A Technical Introduction to Flow-based generative model

Section 1

from which we see that we need ■ c ■ < 1 {\displaystyle \lVert \mathbf {c} \rVert <1} to keep ■ {\displaystyle \ell } real and positive for all y ∈ S n – 1 {\displaystyle \mathbf {y} \in \mathbb {S} ^{n-1}} . The differential volume ratio is given (without derivation) by Boulerice & Ducharme(1994) as:

Section 2

In fact, the set of f gcal {\displaystyle f_{\text{gcal}}} transformations form a group under function composition. The set of f cal {\displaystyle f_{\text{cal}}} transformations form a subgroup. Also see: Dirichlet calibration, which generalizes f gcal {\displaystyle f_{\text{gcal}}} , by not placing any restriction on the matrix, A {\displaystyle \mathbf {A} } , so that invertibility is not guaranteed. While Dirichlet calibration is trained as a discriminative model, f gcal {\displaystyle f_{\text{gcal}}} can also be trained as part of a generative calibration model.

Section 3

where F x lin = ■ M x + c ■ – 1 (I n – y y ′) (M + c x ′) {\displaystyle \mathbf {F} _{\mathbf {x} }^{\text{lin}}=\lVert \mathbf {Mx} +\mathbf {c} \rVert ^{-1}(\mathbf {I} _{n}-\mathbf {yy} ')(\mathbf {M+cx} ')} is the Jacobian of f lin {\displaystyle f_{\text{lin}}} differentiated w.r.t. its input, but not also w.r.t. its parameter.

Section 4

If f gcal {\displaystyle f_{\text{gcal}}} is to be used as a calibration transform, further constraint could be imposed, for example that A {\displaystyle \mathbf {A} } be positive definite, so that (A x) ′ x > 0 {\displaystyle (\mathbf {Ax})'\mathbf {x} >0} , which avoids direction reversals. (This is one possible generalization of a > 0 {\displaystyle a>0} in the f cal {\displaystyle f_{\text{cal}}} parameter.) For n = 2 {\displaystyle n=2} , a > 0 {\displaystyle a>0} and A {\displaystyle \mathbf {A} } positive definite, then f cal {\displaystyle f_{\text{cal}}} and f gcal {\displaystyle f_{\text{gcal}}} are equivalent in the sense that in both cases, log ■ p 1 p 2 ■ log ■ q 1 q 2 {\displaystyle \log {\frac {p_{1}}{p_{2}}}\mapsto \log {\frac {q_{1}}{q_{2}}}} is a straight line, the (positive) slope and offset of which are functions of the transform parameters. For n > 2 , {\displaystyle n>2,} f gcal {\displaystyle f_{\text{gcal}}} does generalize f cal {\displaystyle f_{\text{cal}}} . It must however be noted that chaining multiple f gcal {\displaystyle f_{\text{gcal}}} flow transformations does not give a further generalization, because:

Section 5

f θ – 1 {\displaystyle f_{\theta }^{-1}} is just z 1 = x 1 , z 2 = x 2 – m θ (x 1) {\displaystyle z_{1}=x_{1},z_{2}=x_{2}-m_{\theta }(x_{1})} , and the Jacobian is just 1, that is, the flow is volume-preserving. When n = 1 {\displaystyle n=1} , this is seen as a curvy shearing along the x 2 {\displaystyle x_{2}} direction.

Section 6

Since the Jacobian is injective (full rank: m {\displaystyle m}), a local (not necessarily unique) left inverse, say r x ∗ {\displaystyle r_{\mathbf {x} }^{*}} with Jacobian R x ∗ {\displaystyle \mathbf {R} _{\mathbf {x} }^{*}} , exists such that r x ∗ (e x (x ~)) = x ~ {\displaystyle r_{\mathbf {x} }^{*}(e_{\mathbf {x} }({\tilde {x}}))={\tilde {x}}} and R x ∗ T x = I m {\displaystyle \mathbf {R} _{\mathbf {x} }^{*}\mathbf {T_{x}} =\mathbf {I} _{m}} . In practice we do not need the left inverse function itself, but we do need its Jacobian, for which the above equation does not give a unique solution. We can however enforce a unique solution for the Jacobian by choosing the left inverse as, r x : R n → R m {\displaystyle r_{\mathbf {x} }:\mathbb {R} ^{n}\to \mathbb {R} ^{m}} :

Section 7

To derive the inverse transform, with suitable restrictions on the parameters to ensure invertibility. To derive in simple closed form the differential volume ratio, R f {\displaystyle R_{f}} . An interesting property of these simple spherical flows is that they don't make use of any non-linearities apart from the radial projection. Even the simplest of them, the normalized translation flow, can be chained to form perhaps surprisingly flexible distributions.

Section 8

log ■ (p (x)) = log ■ (p (z 0)) – ∫ 0 T Tr [∂ f ∂ z t] d t {\displaystyle \log(p(x))=\log(p(z_{0}))-\int _{0}^{T}{\text{Tr}}\left[{\frac {\partial f}{\partial z_{t}}}\right]dt}

Section 9

Downsides Despite normalizing flows success in estimating high-dimensional densities, some downsides still exist in their designs. First of all, their latent space where input data is projected onto is not a lower-dimensional space and therefore, flow-based models do not allow for compression of data by default and require a lot of computation. However, it is still possible to perform image compression with them. Flow-based models are also notorious for failing in estimating the likelihood of out-of-distribution samples (i.e.: samples that were not drawn from the same distribution as the training set). Some hypotheses were formulated to explain this phenomenon, among which the typical set hypothesis, estimation issues when training models, or fundamental issues due to the entropy of the data distributions. One of the most interesting properties of normalizing flows is the invertibility of their learned bijective map. This property is given by constraints in the design of the models (cf.: RealNVP, Glow) which guarantee theoretical invertibility. The integrity of the inverse is important in order to ensure the applicability of the change-of-variable theorem, the computation of the Jacobian of the map as well as sampling with the model. However, in practice this invertibility is violated and the inverse map explodes because of numerical imprecision.

Section 10

which maps a conveniently chosen, (n − 1) {\displaystyle (n-1)} -dimensional representation, p ~ {\displaystyle {\tilde {\mathbf {p} }}} , to the embedded manifold. The n -by- (n − 1) {\displaystyle n{\text{-by-}}(n-1)} Jacobian is

Section 11

Nonlinear Independent Components Estimation (NICE) Let x , z ∈ R 2 n {\displaystyle x,z\in \mathbb {R} ^{2n}} be even-dimensional, and split them in the middle. Then the normalizing flow functions are x = [x 1 x 2] = f θ (z) = [z 1 z 2] + [0 m θ (z 1)] {\displaystyle x={\begin{bmatrix}x_{1}\\x_{2}\end{bmatrix}}=f_{\theta }(z)={\begin{bmatrix}z_{1}\\z_{2}\end{bmatrix}}+{\begin{bmatrix}0\\m_{\theta }(z_{1})\end{bmatrix}}} where m θ {\displaystyle m_{\theta }} is any neural network with weights θ {\displaystyle \theta } .